WASP Software Engineering Assignment

Yizhou Xu
KTH Royal Institute of Technology
yizhoux@kth.se

1 Introduction

This assignment reviews the relationship between software engineering and my research. To provide the necessary context, I will first introduce my research topic.

My Research Topic. My research area focuses on the automatic interpretation of football tracking data and the development of representation-learning methods for such data. Football is one of the most widely followed sports worldwide, and the growing availability of advanced tracking technologies has created new opportunities for data-driven performance and tactical analysis. Modern tracking systems use multiple cameras installed around a stadium to record the positions and movements of players and the ball throughout a match. These systems can already capture detailed player skeleton data, providing rich information about body posture and movement. For example, the tracking technology developed by Tracab[1] enables the large-scale collection of high-quality skeleton data from professional football matches. However, despite the increasing volume and richness of the available data, current methods for automatically interpreting and understanding football tracking data remain limited.

My doctoral research aims to develop AI-based methods for understanding skeleton-based football tracking data. Rather than analyzing players in isolation, my research focuses on interpreting the entire football scene, including the positions, movements, and interactions of all 22 players and the ball over time. In particular, I use machine-learning techniques, including Graph Neural Networks and self-supervised learning, to learn informative representations of player behavior, team interactions, and tactical patterns. These representations can subsequently support applications such as game-event recognition, player-action evaluation, tactical analysis, and team-performance assessment.

2 Lecture Principles

2.1 Multi-level Testing in AI Research

Multi-level testing is common in software engineering, but it is less systematic in traditional AI research. In software engineering, testing typically starts with individual functions, gradually expanding to integrated components, the entire system, and finally user-level verification. This structure is crucial for production software because the final system must be able to run reliably for users. In contrast, many AI research projects are primarily experimental, intended only for researchers. To turn the AI research into a product would be another project. Therefore, unusual edge cases may be less important than ensuring that the main data-processing, training, and evaluation pipelines correctly implement the research idea.

However, I have found testing increasingly valuable as AI coding agents, such as Codex and Claude Code, become more involved in my project. These agents can generate substantial parts of functions and pipelines, but the generated code may not always reflect my intended behaviour. Tests provide a practical way to control its quality. For example, unit tests can check individual preprocessing or graph-construction functions, while integration tests can confirm that the data pipeline and model

[1] https://www.ea.com/tracab

work together correctly. In this setting, tests also act as executable specifications of my ideas. They help verify that the agent has implemented what I intended. AI agents can reduce implementation effort, but they do not remove the need to judge whether generated code is correct. Therefore, multi-level testing is a useful software-engineering principle that can be borrowed to improve the reliability of AI research code in the era of AI coding agents.

2.2 The Science-Engineering Loop in my Research

My research project combines scientific investigation with substantial engineering work. From a scientific perspective, I investigate which methods can best learn representations of football player motion from tracking data. However, unlike research based on standardized and preprocessed benchmarks, my work starts from raw data. As a result, a lot of engineering work is required, including processing the raw tracking data, filtering and grouping samples, constructing datasets, creating annotations, and defining appropriate data splits. To support this process, I also developed an annotation software system. These parts must be reliable enough to ensure that the experimental results are trustworthy.

The lecture describes machine learning as both an empirical science and a design discipline, involving an iterative cycle of experimentation, implementation, observation, and refinement. This closely reflects my doctoral project. Although the main goal is to produce strong scientific contributions, the validity of those contributions depends heavily on the quality of the underlying engineering. Errors in data processing, annotation, dataset construction, or evaluation could lead to incorrect scientific conclusions. Therefore, my research is scientific in its objectives but strongly dependent on engineering in its execution.

3 Guest-lecture Principles

3.1 Incremental Refinement of Research Goals

Although research differs from software engineering, both begin with considerable ambiguity. Requirements engineering recognizes that requirements are often initially vague, or ambiguous and should therefore be elicited and refined incrementally rather than defined all at once. In software engineering, this ambiguity mainly concerns what stakeholders need from a system. In research, it concerns how a broad and underexplored problem should be defined.

My research topic is understanding football through AI, which can also be interpreted in many ways. The focus could be an individual player, a groups of players, or the complete scene on the pitch. "Understanding" could refer to tactical patterns, player actions, physical condition, capability, or other aspects of performance. However, these possibilities are not equally feasible or valuable. Because my research is application-oriented, the needs of the football community should also influence how the goal is refined. Relevant outcomes may provide value to coaches, players, clubs, fans, or data companies. This resembles requirements engineering, in which stakeholders and their goals are identified before specific system requirements are developed.

Research differs because the feasibility and value of a goal cannot always be known in advance. An investigation may reveal that a question is too broad, that the available data are insufficient, or that another direction is more promising. Incremental refinement is therefore particularly useful: I can begin with questions that are achievable with the available data, and then adjust the research goal as new knowledge is gained. As the project progresses, the results make the next meaningful research step clearer.

3.2 Agentic AI in AI Research

Agentic AI is transforming software engineering by automating tasks at multiple levels, from implementation and testing to workflow management and decision support. This automation is changing the role of software engineers, shifting their focus from routine execution to higher-level responsibilities such as defining requirements, system design, and strategic planning. A similar evolution is occurring in AI research. From my experience, AI agents like Codex are already capable of implementing well-defined ideas, managing experimental pipelines, and summarizing results. By handling much of

the repetitive engineering work, these tools enable researchers to test and iterate on ideas with greater efficiency.

However, the capabilities of current AI agents are limited when it comes to the core aspects of scientific discovery. They still lack the judgment required to determine the novelty of a research direction, identify the most insightful next step, or assess the elegance and true value of an idea. Therefore, researchers must currently define the direction, formulate clear ideas, and evaluate the scientific significance, while AI supports implementation and experimentation. While these technological limitations may change over time, I believe the role of human direction will remain fundamental. The ultimate goal of AI research is to create tools that serve human needs and values. Therefore, it is essential that humans continue to define which problems are worth solving and which outcomes are truly meaningful.

4 Data Scientists, Software Engineers, and AI Engineers - Q & A

Q: What differences do the chapters describe between data scientists and software engineers? Do you agree with this distinction? Why or why not?

A: The chapters distinguish data scientists and software engineers mainly by the scope and goal of their work. Data scientists are model-oriented: they work with data, model architectures, feature engineering, and hyperparameter tuning, with the main objective of improving model performance. Software engineers are system-oriented: their goal is to build reliable software that satisfies user needs, with the model treated as one component of a larger system. Their work therefore includes requirements analysis, system architecture, implementation, testing, deployment, and long-term maintenance.

I agree with this distinction. They are two jobs with different responsibilities and success criteria. In a company, data scientists generally focus on improving the model, while software engineers integrate that model into a usable and reliable product. Naturally, their work overlaps and requires communication, but this is not unique to machine-learning projects. Software engineers must also communicate to other people like product managers and marketing specialists. The need for communication does not mean that the roles themselves are unclear.

Q: The chapters argue that production ML requires a system-wide view, not only a model-centric view. How does this apply to your own research area?

A: My current research is not a production-ready system and will not directly face users in the near future. Its primary goal is scientific: to investigate effective methods for learning representations from football tracking data. In this context, I believe it is reasonable to remain relatively model-centric. Requirements such as deployment, operational monitoring, and business cost are not yet central to the research question.

Nevertheless, my work still depends on a complete system. A system-wide view is relevant for ensuring scientific validity. An error in data processing, annotation, or evaluation could produce an incorrect scientific conclusion. Software-engineering principles are useful when they improve the reliability of this research pipeline, but not every principle designed for customer-facing production systems is directly applicable at the current stage.

Q: Do you think data scientists and software engineers will remain distinct roles, or will both need to learn more of each other's skills?

A: I believe data scientists and software engineers will remain distinct roles because they have different goals and require different specialist skills. Developing a strong machine-learning model requires deep knowledge in areas such as data, optimization, and model behaviour. Building a reliable software product requires expertise in areas like architecture, testing, deployment, and maintenance. A successful ML-enabled product needs both forms of expertise.

It is useful for data scientists to understand some software engineering and for software engineers to understand the fundamentals of machine learning. However, the main purpose of this is to support communication and cooperation rather than to make everyone capable of performing every role. No individual is expected to know everything. Complex systems are built by specialists who communicate and combine their expertise.

Q: Where does "AI Engineering" fit in this picture: is it mostly software engineering for AI systems, data science made more systematic, MLOps/model engineering, or something partly distinct?

A: "AI Engineering" is a relatively new concept. The book [2] notes that it is often used to describe automated ML pipelines, model deployment, scalable inference, and MLOps, although it may also be used more broadly. My view is that AI Engineering is mainly software engineering applied to systems containing AI components, together with systematic practices for developing those AI components. I do not see it as a fundamentally separate engineering discipline.

A machine-learning model is just a software component with greater uncertainty and less precisely specified behaviour. However, software engineering already has methods for handling unreliable components. AI may require these practices to be adapted or applied more carefully, but the fundamental goal remains unchanged: users still need reliable and useful software. The presence of a machine-learning model should not be used as a reason to lower software quality. Models are introduced to provide better functionality, not a worse user experience. From this perspective, what matters is not inventing new engineering principles, but deciding which existing principles must be preserved when AI components are introduced. For example, MLOps can be understood as a natural extension of DevOps that maintains software quality throughout the machine-learning lifecycle.

Q: How do foundation models, AI coding assistants, RAG systems, prompt pipelines, and agentic workflows change the boundaries between these roles?

A: It is difficult to predict how foundation models and AI agents will ultimately change the boundaries between data scientists and software engineers because their capabilities are developing quickly. A conclusion based only on current tools may become outdated very soon. Nevertheless, based on my experience with current tools such as Claude Code and Codex, AI coding assistants have made it easier for machine-learning researchers to build simple demos and for software engineers to experiment with small models. However, current agents still struggle to independently manage complex projects, whether the goal is to develop a complex model for a large research problem or to build a reliable software system. They often lack the broad understanding needed to decide what the system requires and how its parts should be organized.

Specialist knowledge therefore remains essential for guiding AI agents and ensuring that their work contributes to the intended goal. Role boundaries remain clear, as each specialist's expertise continues to provide distinct value within the workflow, even when AI handles much of the implementation.

5 Paper Analysis

5.1 Paper A

Paper Title: Statistical Confidence in Functional Correctness: An Approach for AI Product Functional Correctness Evaluation [1] (CAIN 2026)

5.1.1 Core Ideas

The paper proposes Statistical Confidence in Functional Correctness (SCFC) for evaluating whether an AI system meets quantitative functional requirements. Rather than relying on a single performance estimate, SCFC considers uncertainty by combining specification limits, stratified sampling, bootstrapping, and a capability index. This provides a more reliable basis for judging whether an AI system is functionally ready for practical use.

5.1.2 Integration into a Larger AI-intensive Project

A relevant AI-intensive project is semi-automated offside technology (SAOT). SAOT uses AI models to capture players' 3D skeletons in real time, identify the moment when the ball is played, and combine this information to generate offside decisions that assist referees. Since these decisions can directly influence the outcome of a match, the system needs to be highly reliable rather than merely achieving good average performance.

SCFC could be applied to evaluate whether the system satisfies clearly defined functional requirements. For example, specification limits could be defined for the accuracy of player-position estimation

or the accuracy of the final offside decision. The evaluation data could also be stratified according to challenging conditions, such as player occlusion and crowded scenes. Bootstrapping could then be used to estimate confidence intervals for the relevant performance metrics, and the capability index could indicate whether the system meets its required performance with sufficient statistical confidence.

This is particularly useful for SAOT because a high average accuracy alone may not provide enough evidence of reliability. A system may perform well overall while being unstable in less common but important situations. SCFC therefore provides a systematic way to connect statistical model evaluation with the functional requirements of a high-reliability AI system.

5.1.3 Relation and Adaptation to my Research

The paper is relevant to my research because practical football AI systems can encounter many corner cases that are difficult to capture with standard machine-learning evaluation. Performance on a fixed test set may show that a model works well on average, but it does not necessarily indicate how reliably the model will behave across different game situations. SCFC therefore provides a useful perspective by evaluating not only model performance, but also the statistical confidence that a required level of performance can be achieved.

SCFC is not directly applicable to my current representation learning research, since there is no single functional requirement or clear definition of what constitutes a "correct" representation. However, it could be applied to downstream tasks built on these representations, such as event recognition and player action evaluation, where measurable requirements can be defined.

More generally, I think reliability is undervalued in machine learning research, where evaluation often focuses mainly on average benchmark performance. Reliability is related to robustness, but the emphasis is slightly different. Robustness usually concerns how a model behaves under perturbations or changes in the input or data distribution, while reliability is more concerned with whether the model can consistently satisfy its expected requirements in practical use. I believe that more reliable AI models could be a very promising research direction for machine learning researchers.

5.2 Paper B

Paper Title: Engineering LLM Powered Multi-Agent Framework for Autonomous CloudOps [3] (CAIN 2025)

5.2.1 Core Ideas

The paper presents MOYA, an LLM-powered multi-agent framework developed for autonomous CloudOps. Instead of relying on a single LLM, MOYA decomposes complex workflows into specialized agents coordinated within a modular framework. The design addresses practical engineering concerns such as heterogeneous data sources, task orchestration, extensibility, security, error mitigation, and the balance between agent autonomy and human control.

5.2.2 Integration into a Larger AI-intensive Project

A larger AI-intensive project related to my research could be an intelligent football-analysis platform for professional clubs. Such a system could combine tracking and skeleton data, video, match statistics, and textual information to support coaches and analysts. Similar to MOYA, the system could be organized as a collection of specialized agents rather than one general-purpose AI component. For example, one agent could retrieve and analyze tracking data, another could detect game events, another could analyze tactical patterns, while an LLM-based agent could retrieve relevant match information and generate explanations or reports for analysts. An orchestrator could select and coordinate these agents according to the user's request.

The ideas in MOYA are particularly useful for such a system because football analysis involves different data sources and specialized tasks that cannot easily be handled by a single model. The paper argues that assigning distinct domains to specialized agents can reduce context overload and make the overall system more modular and extensible. Human control would also remain important. The system could assist coaches and analysts with information and recommendations while leaving important decisions to domain experts.

5.2.3 Relation and Adaptation to my Research

The paper is not directly related to my current representation learning problem, but it provides a useful perspective on how my research could eventually become part of a larger AI system. At present, I mainly focus on developing models that learn meaningful representations from football tracking data. If these models are later used in a practical football analysis platform, they would likely become only one specialized component among many others. The multi-agent architecture in the paper suggests a natural way of combining such specialized AI capabilities while keeping their responsibilities and interfaces separated.

To better support this type of integration in the long term, I could design my research project in a more modular way, with clear interfaces between data processing, representation learning, and downstream analysis. The trained models could expose well-defined inputs and outputs so that they can later be called as tools by other components or agents. These adaptations would not change the scientific goal of my research, but they would make its results easier to reuse as a specialized capability within a larger agent-based football analysis system.

6 Evidence, Originality, and GenAI-use Transparency

See References [1, 2, 3] for main papers and sources I relied on.

[1] and [3] are research papers from CAIN conference. [2] is a book and I use Chapter 1 and 2 as background for the discussion in Section 4.

Generative AI was used only for writing and formatting purposes. The claims, arguments, and opinions presented in this assignment are my own.

References

[1] Wallace Albertini, Marina Condé Araújo, Júlia Condé Araújo, Antonio Pedro Santos Alves, and Marcos Kalinowski. Statistical confidence in functional correctness: An approach for AI product functional correctness evaluation. In *Proceedings of the IEEE/ACM 5th International Conference on AI Engineering – Software Engineering for AI*, CAIN '26, pages 1–11, 2026. doi: 10.1145/3793653.3793770.

[2] Christian Kastner. *Machine learning in production: from models to products*. MIT Press, 2025.

[3] Kannan Parthasarathy, Karthik Vaidhyanathan, Rudra Dhar, Venkat Krishnamachari, Adyansh Kakran, Sreemaee Akshathala, Shrikara Arun, Amey Karan, Basil Muhammed, Sumant Dubey, and Mohan Veerubhotla. Engineering LLM powered multi-agent framework for autonomous cloudops. In *Proceedings of the IEEE/ACM 4th International Conference on AI Engineering – Software Engineering for AI*, CAIN '25, pages 201–211, 2025. doi: 10.1109/CAIN66642.2025.00031.